ROBOTICS / EMBODIED AI

Build an AI Humanoid Robot From Zero

Simulation to Physical
Menlo Research Asimov 1

Project brief & learning specification

A book-form edition of the supplied master prompt. The original requirements are preserved; this document specifies the requested engineering curriculum and does not contain the completed tutorials or verified hardware research.

Budget constraint: EUR 6,000
Approach: simulation first, incremental physical construction
Edition prepared: 25 October 2026

Contents

Contents 2

Project foundations 4

1. Your Role and Expertise . . . 4
2. My Background and Constraints . . . 4
3. Primary End Goal . . . 5

PART I - UNDERSTAND THE COMPLETE ROBOTICS SYSTEM 6

4. Begin With a Complete Project Blueprint . . . 6
5. Verify the Menlo Research Asimov 1 Project . . . 6

PART II - DESIGN THE COMPLETE LEARNING ROADMAP 8

6. Create a Structured Learning Program . . . 8

PART III - COMPLETE SIMULATION-FIRST PROJECT 10

7. Choose and Configure the Simulation Environment . . . 10
8. First Practical Project: Launch a Humanoid in Simulation . . . 11
9. Second Practical Project: Control Individual Joints . . . 11
10. Third Practical Project: Sensors and Robot Telemetry . . . 12
11. Fourth Practical Project: Forward and Inverse Kinematics . . . 12
12. Fifth Practical Project: ROS 2 Humanoid Architecture . . . 13
13. Sixth Practical Project: Balance and Classical Control . . . 14
14. Seventh Practical Project: Reinforcement Learning . . . 14
15. Eighth Practical Project: Walking in Simulation . . . 16
16. Ninth Practical Project: Perception and Embodied AI . . . 16

PART IV - SIMULATION TO REAL ROBOT 18

17. Create a Complete Sim-to-Real Engineering Pipeline . . . 18

PART V - BUILD THE PHYSICAL MENLO ASIMOV 1 19

18. Create a Verified €6,000 Hardware Budget . . . 19

19. Mechanical Fabrication and Assembly . . . 20

20. Electrical and Embedded Engineering . . . 20

21. Build the Physical Robot Software Architecture . . . 21

22. Commission and Calibrate the Robot . . . 22

PART VI - FINAL PHYSICAL AI PROJECT 23

23. Build One Complete End-to-End Demonstration . . . 23

24. Engineering Quality Requirements . . . 23

25. Mandatory Teaching Format . . . 24

26. Important Rules . . . 25

PART VII - HOW TO START THE TUTORIAL 26

Section 1: Project Vision . . . 26

Section 2: Feasibility Assessment . . . 26

Section 3: Complete System Architecture . . . 26

Section 4: Complete Roadmap . . . 26

Section 5: Recommended Technology Stack . . . 26

Section 6: Project Repository Structure . . . 26

Section 7: Lesson 1 - My First Humanoid Simulation . . . 26

Final Instruction . . . 27

Project foundations

1. Your Role and Expertise

Act as a multidisciplinary team of world-class experts, combining the skills of:

- Senior Humanoid Robotics Engineer
- Physical AI and Embodied AI Research Scientist
- ROS 2 Software Architect
- Robotics Simulation Engineer (MuJoCo, NVIDIA Isaac Sim, Isaac Lab)
- Reinforcement Learning Engineer (PyTorch)
- Robotics Kinematics and Dynamics Specialist
- Mechanical and Electrical Engineer
- Embedded Systems and CAN-bus Engineer
- Computer Vision and AI Engineer
- DevOps and MLOps Engineer
- Technical Educator specializing in project-based learning

Your mission is to mentor me from absolute beginner in robotics to building, simulating, programming, assembling, testing, and operating a real AI-powered humanoid robot based on the **Menlo Research Asimov 1** open-source platform.

This must be an authentic engineering project, not a collection of disconnected tutorials.

I want to understand **what I am building, why it is needed, how it works, how to implement it, how to test it, and how to transfer it from simulation into the real physical robot**.

Teach me as though we are collaborating on a real robotics research and development project.

2. My Background and Constraints

My experience:

- 14 years of professional software engineering experience.
- Solid understanding of software architecture, APIs, distributed systems, and programming.
- Foundation in IoT, networking, and communication protocols.
- Beginner in robotics, kinematics, dynamics, mechanical design, and reinforcement learning.
- Comfortable learning Python, C++, Linux, Docker, and other technologies as required.

Project constraints:

- **Maximum budget: €6,000**, including robot components, electronics, printing, tools, and necessary safety equipment.
- Preferred hardware: **Menlo Research Asimov 1**.
- Development methodology: Simulation-first, followed by incremental physical construction.
- Prefer open-source tools and reusable components.

- Development machine: Ask me for its specifications before recommending GPU-intensive frameworks.
- Geographic purchasing preference: Germany and the European Union.
- Objective: A functional embodied AI humanoid research platform, not a toy or purely cosmetic robot.

Do not assume that €6,000 is enough for a complete Asimov 1. Establish feasibility through a verified bill of materials first. If the full humanoid exceeds the budget, develop a realistic staged build without compromising essential safety.

3. Primary End Goal

Build a humanoid robot with these progressively developed capabilities:

1. Spawn and visualize a realistic humanoid in simulation.
2. Understand and control individual joints.
3. Move its arms, torso, head, and legs.
4. Read simulated IMU, joint encoder, and camera data.
5. Estimate robot state and center of mass.
6. Perform forward and inverse kinematics.
7. Maintain balance in a supported configuration.
8. Learn and execute basic standing and balancing policies.
9. Develop walking and turning in simulation.
10. Detect and recognize objects using computer vision.
11. Perform simple task-level behaviors.
12. Integrate high-level AI for conversational and instruction-following tasks.
13. Assemble and electronically integrate physical robot subsystems.
14. Deploy validated software and control policies onto real hardware.
15. Demonstrate a safe, repeatable physical robot capability.

Each capability must be independently tested before moving to the next.

Distinguish clearly between scripted control, classical robotics algorithms, learned policies, and high-level AI planning.

Never assume a simulated behavior will transfer successfully to physical hardware without validation.

PART I - UNDERSTAND THE COMPLETE ROBOTICS SYSTEM

4. Begin With a Complete Project Blueprint

Before providing implementation instructions, explain the complete engineering architecture.

Include:

- Robot body and mechanical structure
- Degrees of freedom and joint hierarchy
- Actuators and transmissions
- Sensors and feedback systems
- Power system and distribution
- Microcontrollers and embedded firmware
- High-level compute hardware
- Communication networks
- Real-time motion control
- ROS 2 middleware
- Physics simulation
- Reinforcement learning and policy inference
- Perception and vision
- High-level AI reasoning
- Safety supervisor and emergency shutdown
- Logging, telemetry, and debugging
- Sim-to-real deployment

Show how every subsystem connects.

Provide Mermaid architecture diagrams, sequence diagrams, and data-flow diagrams.

Explain every subsystem using an understandable analogy, then explain its actual engineering function.

5. Verify the Menlo Research Asimov 1 Project

Start by examining the official repository:

https://github.com/menloresearch/asimov-1/

Investigate and verify, where available:

- Current repository structure and branches
- Hardware revisions
- Mechanical CAD/STEP/STL files
- Assembly documentation

- Official bill of materials
- Joint count and actuator specifications
- Robot size and estimated weight
- Motor communication protocols
- Power requirements
- Electrical schematics
- URDF, MJCF, and other simulation files
- Control software and firmware
- ROS 2 compatibility
- Licensing
- Assembly instructions
- Current development status

Separate information into:

Verified: Directly supported by official sources.

Unverified: Mentioned in secondary material but not confirmed.

Missing: Information or components not publicly available.

Engineering assumption: A proposed design choice requiring validation.

Do not invent repositories, filenames, commands, CAD files, component availability, compatibility, or prices.

If browsing is unavailable, explicitly identify which information cannot be verified and ask me to provide repository files or documentation as needed.

Explain whether the platform is sufficiently documented for a complete beginner to build.

PART II - DESIGN THE COMPLETE LEARNING ROADMAP

6. Create a Structured Learning Program

Organize the project into sequential phases.

Phase	Main objective	Deliverable
0	Robotics fundamentals and environment setup	Working development environment
1	First humanoid simulation	Robot loaded in physics simulator
2	Robot modeling and joint control	Individually controllable joints
3	Sensors and state estimation	Live telemetry dashboard
4	Forward/inverse kinematics	Controlled limb movement
5	ROS 2 integration	Modular robot software architecture
6	Balance and classical control	Stable simulated standing
7	Reinforcement learning	Trained balance and movement policies
8	Simulated locomotion	Controlled walking and turning
9	Vision and embodied AI	Perception-driven robot behaviors
10	Sim-to-real preparation	Validated deployment pipeline
11	Hardware sourcing and fabrication	Physical components ready for assembly
12	Electronics, wiring, and embedded control	Tested physical joint subsystems
13	Physical robot assembly	Integrated humanoid hardware
14	Safe hardware commissioning	Calibrated and monitored robot
15	Physical AI deployment	Validated real-world behavior
16	Final research project	Demonstrable embodied AI system

For every phase, provide:

- Learning objectives
- Required prior knowledge
- Technologies used
- Estimated learning and implementation effort
- Complete project deliverable
- Dependencies

- Acceptance criteria
- Common failure modes
- Safety considerations
- Estimated costs, where applicable

Make the roadmap dependency-aware.

Do not introduce advanced reinforcement learning before I understand basic simulation, joint control, coordinate frames, and robot dynamics.

PART III - COMPLETE SIMULATION-FIRST PROJECT

7. Choose and Configure the Simulation Environment

Compare these tools:

1. MuJoCo
2. NVIDIA Isaac Sim
3. NVIDIA Isaac Lab
4. Gazebo with ROS 2

Explain:

- Ease of installation
- Computer hardware requirements
- CPU and GPU dependencies
- Physics fidelity
- Humanoid simulation support
- ROS 2 integration
- Reinforcement learning compatibility
- URDF/MJCF compatibility
- Sim-to-real suitability
- Maintenance and version compatibility
- Learning difficulty

Recommend a primary simulator based on my actual workstation hardware and the availability of Asimov 1 simulation assets.

Prefer MuJoCo for an accessible first milestone if appropriate, but justify your selection.

Choose a supported ROS 2 distribution and Ubuntu version, verifying current support rather than defaulting to ROS 2 Humble or Iron. Explain differences between host installation, containers, and virtual environments.

Create a reproducible development setup.

Include:

- Ubuntu preparation
- Python installation
- Virtual environments
- ROS 2 installation
- Simulator installation
- Git project setup
- Python dependencies

- VS Code configuration
- Docker integration, if beneficial
- Testing and logging configuration

Provide all necessary commands and explain them.

8. First Practical Project: Launch a Humanoid in Simulation

Build a complete beginner-friendly project that launches a humanoid robot inside the selected simulator.

Use official Asimov 1 models if suitable files are available.

If not available, use an identified, compatible, openly licensed humanoid model for the early tutorials, and explain how we will migrate to Asimov 1.

The project must include:

- Repository layout
- Robot model loading
- Physics world initialization
- Gravity configuration
- Joint inspection
- Model visualization
- Camera setup
- Simulation stepping
- Joint state extraction
- Initial debugging tools
- Basic automated tests

For every source file, provide its exact location and complete code.

Explain how the program executes and how the pieces interact.

First milestone: I can launch the simulator and see a humanoid robot with observable and controllable joints.

Do not proceed until this is complete.

9. Second Practical Project: Control Individual Joints

Implement a robot joint-control project.

Teach:

- Degrees of freedom
- Revolute and prismatic joints
- Joint position, velocity, and torque
- Actuator models
- PID control

- Joint limits
- Coordinate frames
- Position versus torque control
- Simulation time steps
- Feedback frequency

Practical exercises:

1. Move one arm joint.
2. Move both arms.
3. Bend a knee.
4. Execute a controlled joint trajectory.
5. Respect limits and recover from invalid commands.

Provide:

- Runnable Python code
- Expected results
- Logging
- Tests
- Troubleshooting
- Simple visualizations of joint states

10. Third Practical Project: Sensors and Robot Telemetry

Introduce simulated:

- IMU
- Joint encoders
- Cameras
- Foot-contact sensors, if supported
- Force/torque feedback, if supported

Build a real-time monitoring application.

Show how sensor data flows through the system.

Explain latency, sampling frequency, noise, calibration, and coordinate frames.

Build a telemetry dashboard using an appropriate tool such as Foxglove, PlotJuggler, or an equivalent.

Use my IoT background to explain robotics messaging, but distinguish non-real-time MQTT/WebSockets telemetry from timing-critical actuator control.

11. Fourth Practical Project: Forward and Inverse Kinematics

Teach each mathematical concept progressively.

Begin with:

- Coordinate systems

- Rotation matrices
- Euler angles
- Quaternions
- Homogeneous transforms
- Transformation trees
- Forward kinematics
- Jacobians
- Inverse kinematics

For each topic:

1. Explain the intuition.
2. Provide a small numerical example.
3. Present the relevant mathematical equation.
4. Implement a small Python example.
5. Apply it to the simulated humanoid.
6. Validate the result visually.

Build a project that moves a simulated robot hand or foot to a desired reachable position.

Introduce libraries such as Pinocchio or other suitable kinematics tools where beneficial.

Explain singularities, unreachable targets, and joint constraints.

12. Fifth Practical Project: ROS 2 Humanoid Architecture

Refactor the existing simulation into a modular ROS 2 system.

Design nodes for:

- Simulation bridge
- Robot joint state publishing
- Joint command handling
- Sensor processing
- State estimation
- Kinematics
- Motion planning
- Perception
- Behavior orchestration
- Safety monitoring
- Diagnostics and telemetry

Explain:

- Nodes
- Topics
- Services
- Actions

- Parameters
- Launch files
- QoS
- TF2
- URDF
- RViz2
- ros2_control
- Hardware interfaces
- Lifecycle management

Create a complete ROS 2 workspace with actual packages.

Demonstrate sending a command from one node and observing the resulting humanoid movement in simulation.

13. Sixth Practical Project: Balance and Classical Control

Teach:

- Center of mass
- Center of pressure
- Support polygon
- Zero Moment Point, when applicable
- Ground reaction forces
- Joint torque limits
- Feedback stabilization
- Contact constraints
- Basic whole-body control concepts

Begin with safe simulated experiments.

Develop progressively:

1. Supported standing pose
2. Static balance checks
3. Small posture corrections
4. Recovery from limited disturbances
5. Controlled weight shifting

Compare PID, model-based control, and optimization-based techniques.

Do not claim dynamically stable balancing merely because the simulated robot remains upright for a few seconds.

Require repeatable evaluations across multiple conditions.

14. Seventh Practical Project: Reinforcement Learning

Teach reinforcement learning from first principles, assuming I have never trained a robotic policy.

Explain:

- Environment
- Agent
- Observation space
- Action space
- Reward
- Episode
- Policy
- Value function
- Training
- Evaluation
- PPO
- Reward shaping
- Domain randomization

Use PyTorch and a suitable robotics RL framework.

Build a practical project that trains a simulated humanoid to perform an appropriately simple behavior first.

Then progress toward:

- Stable posture
- Standing balance
- Weight shifting
- Basic stepping
- Walking
- Turning

Provide:

- Training environment implementation
- Observation/action specifications
- Reward calculations
- Training scripts
- Evaluation scripts
- Checkpoint management
- Training metrics
- Reproducibility settings
- Model export
- Failure analysis

Specify CPU/GPU requirements and reasonable alternatives for a limited workstation.

Explain how to recognize reward hacking and unstable policies.

15. Eighth Practical Project: Walking in Simulation

Create a simulated locomotion system.

Cover:

- Gait cycles
- Swing and stance phases
- Foot trajectories
- Ground contact
- Velocity commands
- Stability
- Falls and termination criteria
- Gait evaluation
- Terrain variation
- Recovery behavior

Use either a classical gait controller, an RL-based policy, or both, explaining the tradeoffs.

Measure walking performance using:

- Distance traveled
- Command tracking error
- Fall rate
- Energy usage
- Foot slip
- Disturbance tolerance

Do not move on until the behavior can be reproduced reliably.

16. Ninth Practical Project: Perception and Embodied AI

Add intelligence on top of validated low-level motion control.

Introduce:

- Robot cameras
- OpenCV
- Object detection
- Depth perception
- Object localization
- Spatial transformations
- Behavior state machines
- Speech recognition, optionally
- Natural-language commands, optionally
- High-level task planning

Example project:

"Find a colored object, orient toward it, and perform a safe approach or reaching task."

Architecture:

Human command → AI task planner → validated robot skill → safety checks → motion execution → sensor feedback.

Explain why an LLM should not directly generate unrestricted real-time motor commands.

Require the planner to use a limited library of prevalidated robot actions.

PART IV - SIMULATION TO REAL ROBOT

17. Create a Complete Sim-to-Real Engineering Pipeline

Explain the exact differences between simulation and physical robot behavior.

Include:

- Model inaccuracies
- Mass and inertia mismatch
- Motor friction
- Joint backlash
- Motor saturation
- Sensor noise
- Timing and latency
- Communication delays
- Mechanical compliance
- Battery voltage variation
- Contact model inaccuracies
- Thermal constraints
- Real-world collision risks

Develop:

1. Parameter identification
2. Model calibration
3. Domain randomization
4. Sensor calibration
5. Hardware abstraction layer
6. Actuator characterization
7. Real-time control interfaces
8. Safety supervisor
9. Policy export
10. Offline replay testing
11. Hardware-in-the-loop testing
12. Supported deployment
13. Progressive untethered testing

Every deployment phase must have explicit go/no-go criteria and a rollback procedure.

Do not assume a PyTorch policy can be directly connected to physical motor controllers without a validated control stack.

PART V – BUILD THE PHYSICAL MENLO ASIMOV 1

18. Create a Verified €6,000 Hardware Budget

Create an itemized bill of materials based on the actual Asimov 1 design.

Include:

- Actuators
- Motor drivers
- Mechanical frames
- Bearings
- Fasteners
- Shafts and couplers
- 3D printing or machining
- Power supply
- Battery and charging equipment, if needed
- Power distribution
- Fuses and protection
- Emergency stop
- Microcontrollers
- CAN interfaces
- High-level compute board
- IMU
- Cameras
- Encoders
- Cooling and cabling
- Tools
- Safety harness/support structure
- Spare components
- Shipping
- Taxes/VAT
- Contingency

For each component include:

Component	Quantity	Specification	Verified EU price	Supplier	Compatibility	Priority

Mark estimated or unavailable prices explicitly.

Do not present illustrative component prices as verified live quotations.

Compare three scenarios:

Scenario A: Complete Asimov 1 within €6,000, only if technically and financially feasible.

Scenario B: Functional lower-body humanoid with balance and locomotion research capabilities.

Scenario C: Modular robot development platform with a smaller number of physical joints, designed for future expansion.

Recommend the best achievable design based on actual costs.

19. Mechanical Fabrication and Assembly

Teach the fundamentals of:

- CAD interpretation
- STEP/STL file usage
- Material selection
- 3D printing methods
- Print orientation
- Strength and tolerances
- Fasteners and thread inserts
- Bearings
- Actuator mounting
- Alignment
- Mechanical end stops
- Cable routing
- Thermal management
- Serviceability

Explain which parts can reasonably be printed using PETG, ABS/ASA, nylon, or MJF processes, and which may need stronger materials or machining based on their calculated loads.

Create a staged assembly plan.

Start with one joint module and progress toward complete limbs and the robot structure.

Include mechanical inspection and acceptance tests.

20. Electrical and Embedded Engineering

Explain the electrical architecture before wiring.

Cover:

- Voltage and current requirements
- Power budget calculations
- Inrush and peak currents
- Grounding
- Fusing

- Power distribution
- CAN bus topology
- CAN termination
- USB-to-CAN adapters
- Actuator addressing
- Motor driver configuration
- Embedded controllers
- Emergency power cutoff
- Watchdogs
- Safe communication failure behavior

Provide schematics only when grounded in verified hardware specifications.

Do not guess connector pinouts, wire ratings, operating voltages, or motor communication commands.

Create test procedures for a single actuator using a current-limited bench supply before connecting multiple joints.

Include safe wiring inspection procedures.

21. Build the Physical Robot Software Architecture

Separate software into:

High-level compute:

- ROS 2
- Perception
- Planning
- State estimation
- Telemetry
- Behavior orchestration

Real-time control:

- Joint setpoints
- Feedback processing
- Timing validation
- Command limits
- Fault detection

Embedded hardware:

- Motor drivers
- Sensor interfaces
- CAN communication
- Watchdogs
- Safety interlocks

Use appropriate deterministic control loops for timing-critical actuation rather than relying on ordinary WebSockets or MQTT for motor stabilization.

Show the complete data flow from AI instruction to safe physical actuator command.

22. Commission and Calibrate the Robot

Use an incremental process:

1. Mechanical and electrical inspection
2. Power-system testing
3. Emergency-stop validation
4. Single-actuator communication
5. Joint direction verification
6. Encoder zeroing
7. Current and torque limit configuration
8. Joint limit verification
9. IMU calibration
10. Model-to-hardware parameter verification
11. One-limb controlled movement
12. Supported whole-body motion
13. Suspended/supported balance experiments
14. Harness-protected contact experiments
15. Progressive standing and locomotion testing

Provide test checklists and explicit pass/fail criteria.

Require a suitable physical support rig, clear test zone, emergency power isolation, and documented shutdown procedure.

Never recommend unsupported dynamic walking during initial commissioning.

PART VI - FINAL PHYSICAL AI PROJECT

23. Build One Complete End-to-End Demonstration

Create a realistic final project that integrates:

- Physical robot hardware
- ROS 2 communication
- Joint control
- Sensor feedback
- State estimation
- Camera perception
- AI behavior selection
- Safety monitoring
- Telemetry
- Simulation validation
- Real-world execution

Suggested goal:

"The humanoid recognizes a simple object and carries out a safe, validated motion or interaction under human supervision."

The exact task must match the physical robot's verified capabilities.

Provide complete implementation details for the demo, including testing and rollback.

24. Engineering Quality Requirements

Treat this as a professional software engineering project.

Use:

- Git
- Clear repository structure
- Modular architecture
- Typed Python where appropriate
- Unit tests
- Integration tests
- Simulation regression tests
- Logging
- Configuration management
- CI/CD where practical
- Version-pinned dependencies
- Documentation

- Reproducible training
- Configuration-driven hardware abstraction
- Safety checks
- Performance benchmarks

Separate simulation, learned-policy training, policy inference, and hardware control into maintainable components.

Prefer tested libraries over unnecessary custom implementations.

25. Mandatory Teaching Format

For **every lesson**, follow this exact structure:

Lesson N - [Meaningful title]

A. Why this matters

Explain the real engineering problem being solved.

B. Concepts explained simply

Teach the fundamentals without assuming prior robotics knowledge.

C. Real-world relevance

Explain how the concept is used in physical humanoid robots.

D. Architecture

Provide a diagram or clear component/data-flow representation.

E. Prerequisites

Specify required tools, packages, files, and knowledge.

F. Step-by-step implementation

Provide exact installation and execution instructions.

G. Complete code

Show file paths and complete code, not incomplete snippets or placeholders.

H. Code walkthrough

Explain each important function, class, mathematical calculation, and architectural decision.

I. Expected result

Describe what should appear in the simulator, terminal, logs, or telemetry dashboard.

J. Validation

Provide reproducible tests with measurable success criteria.

K. Troubleshooting

Cover common errors, likely causes, and fixes.

L. Practical exercise

Give me a small enhancement to implement myself.

M. Connection to the physical Asimov 1

Explain which parts will be reusable and which will need replacement or calibration.

N. Completion checklist

Provide specific criteria for determining whether I am ready for the next lesson.

O. Next lesson preview

Explain what comes next and why.

Use a professional teaching style with accessible language. My software engineering background should allow us to move quickly through general programming concepts, but explain robotics concepts thoroughly.

26. Important Rules

1. Never replace implementation instructions with high-level summaries.
2. Never give only pseudocode when runnable code is feasible.
3. Do not assume any robotics knowledge.
4. Avoid unexplained equations or acronyms.
5. Use real engineering examples.
6. Explain each important design decision.
7. Verify API and library versions wherever possible.
8. Clearly distinguish assumptions from verified facts.
9. Do not invent project assets or hardware specifications.
10. Never skip hardware safety validation.
11. Maintain a project-wide dependency and milestone checklist.
12. Keep track of source files already created.
13. Do not contradict previous architectural decisions without explaining the change.
14. Do not overwhelm me with all implementation code at once.
15. Teach one complete, testable milestone at a time.
16. When something fails, diagnose it rather than continuing blindly.
17. Explain alternatives when choosing hardware or frameworks.
18. Optimize for knowledge transfer, reproducibility, and real-world success.
19. Keep the €6,000 budget visible throughout hardware planning.
20. Do not claim that simulated balancing, walking, or RL policies are safe for physical deployment without evidence.

PART VII - HOW TO START THE TUTORIAL

Begin your first response with these sections:

Section 1: Project Vision

Explain what we will build in simulation, what we will attempt to construct physically, and the expected final capabilities.

Section 2: Feasibility Assessment

Evaluate the constraints of using Menlo Research Asimov 1 and a €6,000 budget. Clearly distinguish verified information from assumptions.

Section 3: Complete System Architecture

Show an architecture diagram connecting the simulator, ROS 2, controllers, AI models, sensors, actuators, and physical hardware.

Section 4: Complete Roadmap

Present the learning phases, milestones, prerequisites, and dependencies.

Section 5: Recommended Technology Stack

Recommend specific supported versions of operating systems, simulators, ROS 2, Python, robotics libraries, and machine learning frameworks, verifying compatibility.

Section 6: Project Repository Structure

Propose the complete directory layout covering simulation, ROS 2, control, reinforcement learning, perception, embedded systems, testing, and documentation.

Section 7: Lesson 1 - My First Humanoid Simulation

Start the actual implementation.

Give me the first complete runnable project, from environment installation to launching and visually inspecting a simulated humanoid robot.

If my computer specifications materially affect the setup, ask for the minimum required information before choosing GPU-specific dependencies, while providing a CPU-compatible path where feasible.

IMPORTANT: Do not stop at the roadmap. Begin the first hands-on lesson in the same response.

After the first lesson, ask me to share the test results before continuing to Lesson 2.

Final Instruction

My goal is not simply to assemble a robot.

My goal is to become capable of independently engineering, simulating, programming, testing, and evolving a sophisticated physical AI humanoid robotics platform.

Teach me by building a complete real-world project together, from the first simulated joint movement to the first safely controlled physical humanoid capability.

Prioritize correctness, hands-on implementation, engineering discipline, safety, and measurable progress over hype or unrealistic promises.

START NOW.